

📁 File Management in C — Defining & Opening a File

📖 Definition

File management lets a C program store data **permanently** on disk and read it back later — unlike ordinary variables, which live only in RAM and vanish the moment the program ends. C handles every open file through a special pointer called a **FILE pointer**.

❓ Why Do We Need It?

All the arrays and structures you've used so far are stored in RAM, which is **volatile** — everything is wiped out the instant the program terminates. To keep data around across multiple runs (a saved report, a log, a database of records), it must be written to a **file** on permanent storage. C's `<stdio.h>` library gives one consistent set of functions for this, whether the file is text or binary.

🔗 Defining & Opening a File — fopen()

Every file operation begins by declaring a `FILE` pointer and opening the file with `fopen()` :

```
FILE *fp;           // declare a FILE pointer
fp = fopen("data.txt", "r"); // open "data.txt" for reading
```

Part	Meaning
"data.txt"	The name (and optionally path) of the file to open
"r"	The mode — how you intend to use the file (covered in full next page)
Return value	A working <code>FILE*</code> on success, or <code>NULL</code> if opening failed

⚠ Always check whether `fp == NULL` before using it. Reading or writing through a `NULL` file pointer is undefined behaviour and will typically crash the program.

📄 Example Program

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("data.txt", "w"); // open for writing

    if (fp == NULL) {
        printf("Error: could not open file!\n");
        return 1;
    }

    printf("File opened successfully.\n");
    fclose(fp); // always close what you open
    return 0;
}
```

▶ Output

```
File opened successfully.
```

📌 Note

- The `FILE` pointer returned by `fopen()` must be passed to **every** later operation on that file — reading, writing, and eventually closing it.
- If no path is given, the file is looked for (or created) in the program's current working directory.

💡 Fun Tip

fopen() is like telling a librarian a book's name and how you plan to use it 📖 — she hands you a claim ticket (the **FILE pointer**) you show for every request after that!

🔑 File Opening Modes & Closing a File

📖 File Opening Modes — Definition

The second argument to `fopen()` — the **mode** string — tells C exactly how you intend to use the file: read existing content, overwrite it, or add onto the end.

Mode	Meaning	If File Doesn't Exist	If File Already Exists
"r"	Read only	<code>fopen()</code> fails (NULL)	Reads from the start
"w"	Write only	Creates a new file	⚠️ Erases all existing content
"a"	Append (write only)	Creates a new file	Keeps content; new writes go to the end
"r+"	Read + write	<code>fopen()</code> fails (NULL)	Reads/writes from the start
"w+"	Read + write	Creates a new file	⚠️ Erases all existing content
"a+"	Read + append	Creates a new file	Keeps content; writes always go to the end

⚠️ Opening an existing file in `"w"` mode **instantly erases** everything already in it — a very common beginner mistake. Use `"a"` instead if you want to keep the existing data. Add a `b` to any mode (e.g. `"rb"` , `"wb"`) to open the file in **binary** mode instead of text mode.

📖 Closing a File — `fclose()`

`fclose(fp)` closes an open file — it flushes (physically writes out) any buffered data still waiting in memory, and frees the `FILE` pointer for reuse. Forgetting to close a file can lose that buffered data and leaks a limited OS resource.

```
fclose(fp); // returns 0 on success, EOF on failure
```

📄 Example Program — Modes and Closing in Action

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("log.txt", "w"); // "w" — starts fresh
    fprintf(fp, "First line\n");
    fclose(fp); // MUST close before reopening

    fp = fopen("log.txt", "a"); // "a" — adds onto existing content
    fprintf(fp, "Second line\n");
    fclose(fp);

    fp = fopen("log.txt", "r"); // "r" — read it all back
    char buffer[100];
    while (fgets(buffer, 100, fp) != NULL) {
        printf("%s", buffer);
    }
    fclose(fp);
    return 0;
}
```

► Output

```
First line
Second line
```

🔖 **Fun Tip**

r/w/a are like opening a notebook 📖: **r** = only read what's there; **w** = rip out every page and start blank; **a** = flip straight to the last page and keep writing!

📖 Definition

Once a file is open, C offers several families of functions to actually read and write its content — one character at a time, one line at a time, formatted, or as raw blocks of bytes. This page covers character and string (line) I/O.

🔗 Character I/O — `fputc()` / `fgetc()`

```
fputc(char_value, fp);    // writes ONE character to the file
int ch = fgetc(fp);       // reads ONE character (as an int!)
```

`fgetc()` returns an `int`, not a `char` — so that it can also return the special value `EOF`, which no actual character value could represent.

🔗 String (Line) I/O — `fputs()` / `fgets()`

```
fputs(str, fp);           // writes a string – does NOT add '\n' automatically
fgets(buffer, size, fp);   // reads one line, up to (size-1) characters
```

`fgets()` keeps the trailing `'\n'` if it fits within `size`, and always null-terminates `buffer` for you.

Function	Reads/Writes	Unit
<code>fputc()</code> / <code>fgetc()</code>	Writes / Reads	A single character
<code>fputs()</code> / <code>fgets()</code>	Writes / Reads	A string / a whole line

📄 Example Program

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("note.txt", "w");
    fputs("Hello, File!\n", fp);
    fputc('A', fp);
    fclose(fp);

    fp = fopen("note.txt", "r");
    char ch = fgetc(fp);
    printf("First character: %c\n", ch);

    rewind(fp);                // back to the start (more on this soon)
    char line[50];
    fgets(line, 50, fp);
    printf("First line: %s", line);
    fclose(fp);
    return 0;
}
```

▶ Output

```
First character: H
First line: Hello, File!
```

⚠ Since `fgetc()` returns an `int`, always store its result in an `int` variable — not a `char` — when comparing against `EOF`, or the comparison can silently fail on some systems.

💡 Fun Tip

`fgetc/fputc` pass letters one at a time through a mail slot 📧. `fgets/fputs` mail a whole envelope (line) at once 📧!

📖 Formatted I/O — fprintf() / fscanf()

These work exactly like `printf()` / `scanf()` , except the first argument is a `FILE*` telling them **where** to write to or read from, instead of the screen/keyboard.

```
fprintf(fp, "%s %d\n", name, age);    // write formatted data to a file
fscanf(fp, "%s %d", name, &age);     // read formatted data back from a file
```

📖 Block I/O — fread() / fwrite() (Binary)

These transfer **raw bytes** directly, without converting to/from text — ideal for saving whole structures or arrays quickly, as binary files.

```
fwrite(ptr, size, count, fp);    // write 'count' elements of 'size' bytes each, from ptr
fread(ptr, size, count, fp);     // read 'count' elements of 'size' bytes each, into ptr
```

📄 Example Program — Writing & Reading a Whole Structure

```
#include <stdio.h>

struct Student { char name[20]; int marks; };

int main() {
    struct Student s1 = {"Amit", 88};

    FILE *fp = fopen("student.dat", "wb");    // binary write
    fwrite(&s1, sizeof(struct Student), 1, fp);
    fclose(fp);

    struct Student s2;
    fp = fopen("student.dat", "rb");    // binary read
    fread(&s2, sizeof(struct Student), 1, fp);
    fclose(fp);

    printf("Name: %s, Marks: %d\n", s2.name, s2.marks);
    return 0;
}
```

▶ Output

```
Name: Amit, Marks: 88
```

Function	Purpose	Typical File Type
<code>fprintf()</code> / <code>fscanf()</code>	Formatted, human-readable data	Text files
<code>fread()</code> / <code>fwrite()</code>	Fast, raw block transfer	Binary files

📌 Note

- `fread()` / `fwrite()` work with raw bytes — the resulting file isn't human-readable, and may not be portable across different compilers or systems (padding, endianness can differ).

💡 Fun Tip

fprintf/fscanf write a neatly formatted letter 📄. fread/fwrite ship a sealed box of raw data 📦 — fast, but you can't read what's inside just by looking!

🚨 Error Handling During I/O Operations

📖 Definition

File operations can fail for many reasons — a missing file, a full disk, denied permissions, or the wrong mode. A reliable program always **checks** for these outcomes rather than assuming every read or write succeeded.

① Checking fopen()'s Return Value

The single most important check — always confirm the file actually opened before using it:

```
if (fp == NULL) { /* handle the error */ }
```

② feof() — Detecting End of File

Returns non-zero (true) once the read position has moved **past** the last byte of the file.

⚠ Don't check `feof()` **before** reading to decide whether to read — it only becomes true **after** a read has already failed. Check the return value of the read itself first, then use `feof()` to find out *why* it stopped.

③ ferror() — Detecting a Genuine Error

Returns non-zero if an actual error occurred on the stream — this is how you tell "ran out of data" (`feof`) apart from "something actually went wrong" (`ferror`).

④ perror() — Printing a Human-Readable Message

Prints your own custom message together with the system's description of the last error that occurred.

📄 Example Program

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("missing.txt", "r"); // this file doesn't exist

    if (fp == NULL) {
        perror("Error opening file"); // prints our message + system reason
        return 1;
    }

    fclose(fp);
    return 0;
}
```

▶ Output

```
Error opening file: No such file or directory
```

Function	Purpose
<code>feof(fp)</code>	True once the end of the file has been reached
<code>ferror(fp)</code>	True if a genuine read/write error occurred
<code>perror(msg)</code>	Prints <code>msg</code> plus a description of the last error

⚠ A very common bug is looping with `while (!feof(fp))` — this often processes the last chunk of data **twice**. Prefer checking the return value of `fgets()` / `fscanf()` / `fread()` directly as the loop condition instead.

🌟 Fun Tip

fopen() returning NULL is like reaching for a book that isn't on the shelf 📖 — always check for the empty space before you start reading pages that don't exist!

📖 Sequential vs Random Access Files

📖 Definition

By default, files are read and written **sequentially** — from beginning to end, in order, like reading a book cover to cover. **Random access** lets a program jump directly to any position in the file instead, like flipping straight to a specific page.

📖 Sequential Access — the Default

Every read or write automatically moves the file's internal **position indicator** forward. All the examples on the previous pages used sequential access without needing to say so explicitly.

🔗 Random Access — fseek(), ftell(), rewind()

```
fseek(fp, offset, whence); // jump to a new position
long pos = ftell(fp);      // report the CURRENT position
rewind(fp);                // jump back to the very beginning
```

whence Value	Meaning
SEEK_SET	Offset counted from the beginning of the file
SEEK_CUR	Offset counted from the current position
SEEK_END	Offset counted from the end of the file

📖 Example Program

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("data.txt", "w");
    fputs("0123456789", fp);
    fclose(fp);

    fp = fopen("data.txt", "r");
    fseek(fp, 5, SEEK_SET);           // jump directly to the 6th byte
    printf("Byte at position 5: %c\n", fgetc(fp));

    fseek(fp, 0, SEEK_END);           // jump to the very end
    printf("File size = %ld bytes\n", ftell(fp));

    rewind(fp);                       // jump back to the start
    printf("Byte at position 0: %c\n", fgetc(fp));

    fclose(fp);
    return 0;
}
```

► Output

```
Byte at position 5: 5
File size = 10 bytes
Byte at position 0: 0
```

📖 Note

- ftell() called right after fseek(fp, 0, SEEK_END) is a common trick to find a file's total size in bytes.
- rewind(fp) is exactly equivalent to fseek(fp, 0, SEEK_SET) , just shorter to write.

🌟 Fun Tip

Sequential access is reading a scroll top to bottom 📜. Random access is a bookmark that can teleport straight to any page with fseek() 📖!

] Command Line Arguments

Definition

Command line arguments let a user pass information **into** a C program at the exact moment it's launched from the terminal, instead of the program having to ask for input after it starts running.

🔗 Syntax — A Special main()

```
int main(int argc, char *argv[]) {  
    // ...  
}
```

Parameter	Meaning
argc	"Argument count" — total items typed on the command line, including the program's own name
argv	"Argument vector" — an array of strings; <code>argv[0]</code> is the program's name, <code>argv[1]</code> is the first real argument, and so on

⚠ Every command-line argument arrives as a **string** (`char*`), even if it looks like a number. Convert it yourself with functions like `atoi()` or `atof()` before using it as a number.

📄 Example Program

Suppose the compiled program is run from the terminal as: `./greet Amit 20`

```
#include <stdio.h>  
#include <stdlib.h>  
  
int main(int argc, char *argv[]) {  
    if (argc < 3) {  
        printf("Usage: %s <name> <age>\n", argv[0]);  
        return 1;  
    }  
  
    printf("Program name: %s\n", argv[0]);  
    printf("Name given: %s\n", argv[1]);  
  
    int age = atoi(argv[2]);          // convert the string "20" to an int  
    printf("Age given: %d (in 5 years you'll be %d)\n", age, age + 5);  
    return 0;  
}
```

► Output (running: `./greet Amit 20`)

```
Program name: ./greet  
Name given: Amit  
Age given: 20 (in 5 years you'll be 25)
```

📌 Note

- `argc` is always **at least 1** — the program's own name is counted even if the user supplies no extra arguments.
- `argv[argc]` is guaranteed to be `NULL` , marking the end of the argument list.

🎁 Fun Tip

argc/argv are a delivery manifest handed to your program at launch 📦 — argc says how many packages arrived, argv is the shelf holding each one, starting with the truck's own license plate at argv[0]!